

Conditions for Direct Behavior Specification in Multi-Agent Reinforcement Learning

Yousuf Rashid
University of Waterloo
Waterloo, Ontario, Canada
y2rashid@uwaterloo.ca

Abstract

Direct behavior specification via constrained reinforcement learning replaces hand-tuned reward weights with target rates that Lagrange multipliers enforce on their own. The method was developed and evaluated with off-policy SAC on single-agent navigation tasks. We test it where none of that holds: ten agents sharing one policy, on-policy PPO, an opponent that adapts, and a continuous physics simulation fit to real match tracking data. One constraint transfers. Across a five-target sweep the achieved movement rate sits within about three points of the requested one, and the constrained policy reaches the unconstrained baseline’s success rate on a quarter of the training steps. A second constraint, on crowding near the target zone, does not transfer at any threshold we swept, because the policy already satisfies it at initialization and the multiplier never activates. The original method notes in passing that its threshold semantics assume a binding constraint; we give the first empirical account of what happens when that assumption fails, and the check that catches it beforehand. An ablation gives the mirror failure: computing the indicator per team rather than per agent collapses the weight on task reward and wrecks tracking. We also report a budget-matched baseline that removes a 15-point success cost we had attributed to stacking constraints.

1 Introduction

Reward engineering is how most applied reinforcement learning projects encode what an agent should and should not do. A practitioner writes a weighted sum of terms, trains, watches the resulting behavior, adjusts the weights, and repeats. The weights carry no interpretable meaning, the search over them is expensive, and the agent will exploit any term that admits a shortcut.

Roy et al. [8] propose an alternative. Rather than assigning a weight to a behavior, the practitioner states a target rate for it, and a Lagrange multiplier finds the weight that achieves that rate. Their framework restricts cost functions to indicators, so each threshold reads as a probability the designer can reason about directly, and it handles several constraints at once through multiplier normalisation and a bootstrap constraint on task success. They evaluate it with off-policy SAC on single-agent navigation tasks carrying up to five simultaneous behavioral constraints.

The regime matters because it is where the method would actually be used. The motivation for direct behavior specification is a designer who wants an agent to behave a particular way and does not want to discover the weights by trial and error. In games that agent is rarely alone. It is one of a squad, a patrol or a team, sharing a policy for reasons of memory and training cost, acting against an opponent that is itself reacting, in a simulation with real physics rather than a grid. Each of those properties changes what a

constraint can do. A shared policy means one specification governs ten different roles. An opponent that reacts means the behavior rate the constraint measures moves for reasons the agent does not control. On-policy training means the multiplier and the policy update from the same samples. None of this is exotic, and all of it is outside what the method has been shown to handle.

We test whether the framework survives a regime that shares none of its original algorithmic or structural assumptions. Our environment places ten attacking agents under a single shared policy against a defending side that adapts during training, in a continuous physics simulation calibrated against real match tracking data, trained on-policy with PPO. We report four findings.

- The framework transfers for a constraint the policy violates. Across five target rates at three seeds each, the achieved rate tracks the requested one with a slope of 0.89 and a mean absolute error of 1.5 points on target means, or 4.4 points per run under a frozen-policy probe (Section 6.1).
- A second constraint, on crowding, fails to calibrate at every threshold we swept. Roy et al. note that their threshold semantics hold assuming the constraint is binding; we give the first empirical account of what happens when it is not, and the check that catches it in advance (Section 6.2).
- In a shared-policy setting, the unit the indicator counts over decides whether the constraint works at all. Aggregating over the team rather than per agent collapses the weight on task reward to under one percent and destroys tracking. This design choice has no single-agent analogue (Section 6.4).
- A budget-matched baseline removes a 15-point success cost we had previously attributed to constraint stacking. We report the confound because the experimental design that produced it is a natural one (Section 6.3).

2 Related Work

Constrained MDPs and Lagrangian methods. Constrained MDPs [2] extend the MDP to a policy that maximizes return subject to bounds on auxiliary cost functions. Two families of solution method dominate. Lagrangian methods fold the constraints into a single objective and search for a saddle point, as in reward-constrained policy optimization [12], while projection methods take an unconstrained step and map the policy back into the feasible set, as in constrained policy optimization [1]. Both were developed for safety, where the constraint must hold throughout exploration and the quantity of interest is cumulative violation during training.

The Lagrangian family has a known failure mode that is directly relevant to us. The multiplier update is integral control on the constraint error, so it accumulates while a constraint is violated

and overshoots once the policy crosses the threshold. Stooke et al. [11] add derivative and proportional terms to damp this. We use the plain integral form that Roy et al. use, and the multiplier traces in Section 6.2 should be read with that in mind.

Direct behavior specification. Roy et al. [8] turn the constrained MDP into an interface for designers rather than a safety mechanism. Their three components are the ones we restate in Section 4: cost functions restricted to indicators, so each threshold is a probability between zero and one; multipliers normalized on a simplex, so no constraint can dominate the update without bound; and a bootstrap constraint on task success, which lends its multiplier to the main objective while the optimizer is still searching for a feasible policy.

Their evaluation is single-agent navigation in two environments, an arena with lava, energy and look-at-marker constraints and a larger open world, with up to five behavioral constraints active at once. The learner is SAC, replicated with TD3. The hazards do not move and the agent has no opponent. Our contribution is not to add constraints beyond their five. It is to ask whether the interface still behaves as an interface when the agent is one of ten sharing a policy, when training is on-policy, and when the environment pushes back.

Safe multi-agent RL. Constrained reinforcement learning has been extended to multiple agents. Gu et al. [4] formulate safe MARL as a constrained Markov game and give two algorithms, MACPO and MAPPO-Lagrangian, with monotonic improvement and constraint satisfaction guarantees per iteration, evaluated on safe multi-agent MuJoCo and robosuite benchmarks. Earlier, CMIX [7] handled peak cost constraints by modifying the reward under a QMIX-style mixer.

That line and ours differ in what the constraint is for. Those constraints encode safety, they are chosen by the benchmark rather than by a designer, and the threshold is a budget to stay under rather than a rate to hit. Our thresholds are specifications: a designer names a behavior and a frequency, and the value of the method is whether the agent then exhibits that behavior at that frequency. Overshooting a safety budget in the safe direction is a success; overshooting a specification in either direction is a failure. Our crowding result in Section 6.2 is a case that safe MARL would score as fully satisfied and that a designer would score as broken.

Emergent side effects. Constraining one behavior can produce another that nobody specified, which is the negative side effects problem of Amodei et al. [3]. Two lines of work respond to it by penalizing an agent for changing the world more than the task requires, either by measuring reachability of states the agent could otherwise have reached [5] or by measuring loss of the agent’s own attainable utility across auxiliary reward functions [13]. Those methods define the penalty in advance and do not require the designer to name the side effect. The workflow we study is the opposite one: observe the side effect after the fact, name it, and add a constraint on it. Section 6.2 is about why that second step can fail without saying so.

Reinforcement learning in football. Football is a standing multi-agent benchmark. Google Research Football [6] provides a full eleven-a-side simulator with either one agent controlling the team or one per player, and is where most multi-agent RL work on the

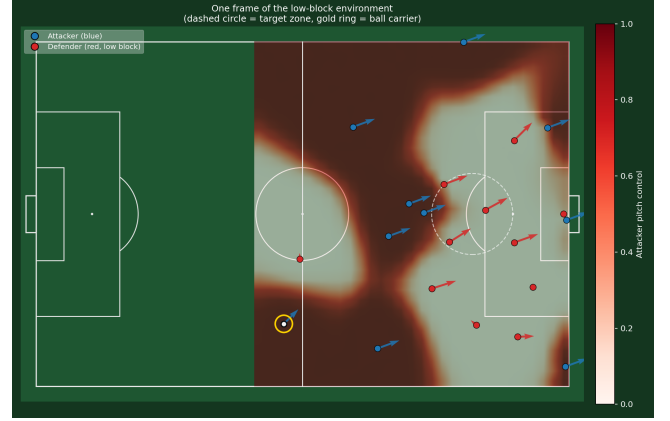


Figure 1: A single tick of the environment. Ten attackers (one holding the ball) face eleven defenders in a 5-4-1 low block. The shaded region is the attacking side’s pitch control, computed on a 31×34 grid of 2 m cells. The circle at (86, 34) is the target zone; success requires the carrier inside it with mean attacking control over its cells of at least 0.45.

sport is run. Separately, TacticAI [14] shows that models built on real tracking data are accepted by practitioners, and it uses the same pitch control formulation we build our reward on.

We use a custom environment rather than Google Research Football for one reason. Our experiments need behavioral indicators that are cheap to measure per agent per tick, thresholds that can be swept, and a physics model whose parameters are fitted rather than fixed, and we need to be able to disable one constraint and rerun. A general-purpose game simulator makes each of those harder than writing the scenario directly.

3 Environment and Task

3.1 The low-block scenario

Ten attackers share a single policy against eleven defenders holding a 5-4-1 low block on a 105×68 m pitch. The attacking side starts in its own half with possession and must work the ball into a target zone, a disc of radius 8 m centred at (86, 34), in front of the defending goal.

An episode succeeds when the ball carrier is inside the zone and the attacking side holds mean pitch control of at least 0.45 over the zone’s cells. Both conditions matter. Reaching the zone with the ball is not enough if the defense has collapsed around it, which is what makes the task about creating space rather than about navigation. An episode fails on a turnover, either a lost ground duel, an intercepted pass, or an offside, and times out after 200 ticks at 0.1 s per tick, or 20 seconds of simulated play.

The reward is sparse at termination, +1.0 for success, -0.3 for a turnover and -0.5 for a timeout, with dense potential-based shaping on the ball’s distance to the zone centre. The potential is $\phi = -0.03 \|p_{\text{ball}} - c_{\text{zone}}\|$ and is zeroed at termination so the shaping telescopes and leaves the optimal policy unchanged.

The low block is the configuration worth studying because it removes the easy solution. Against a high line the attackers can

play direct and beat the defense with pace. Against a deep block there is no space behind, so the task can only be solved by moving defenders, which requires the ten agents to do different things at the same time. A shared policy has to produce that differentiation from identical weights, and the behavioral constraints we impose act on exactly the movement patterns that produce it.

The defending side is not learned. Its block depth tracks the attackers’ four deepest players and the ball’s x position, and its lateral centroid follows the ball with fitted gains, so the defense responds to the attack within an episode without changing across training. This keeps the opponent fixed as an experimental control while still presenting a moving target within an episode.

3.2 Physics and calibration

Players integrate at 0.1 s under a maximum speed of 5 m/s and a maximum acceleration of 7 m/s². Space is scored with the pitch control model of Spearman et al. [9, 10], which integrates for every player at every grid point the probability that the player arrives and controls the ball first. We evaluate it on a 31×34 grid of 2 m cells covering the attacking 62×68 m of the pitch.

Four components are fitted to tracking data rather than chosen by hand, each with a held-out check reported in the corresponding script. Carrying speed is 0.81 of free-running speed, taken as the median per-player ratio of 95th percentile peak speed on carrying runs to peak speed on ball-free runs, over 124 carrying runs and 2957 free runs from twelve players, restricted to efforts above 4 m/s. Only the ratio transfers, since real players top out near 9 m/s against our cap of 5. Pass speed scales with pass length, fitted to 793 passes and binned by length, replacing the constant 15 m/s the engine used before. Defender formation parameters are fitted per slot and per frame from labelled freeze frames, with a resampling procedure that refits on a subsample and evaluates on frames the fit never saw. Starting formations for both sides are drawn from a correlated Gaussian fitted the same way. The control threshold that defines success, 0.45, is the 25th percentile of the control level real shots were taken from, so it admits the situations three quarters of real shots occurred in.

Two of these fits are worth reporting as negatives. The first anchor we tried for the defender calibration, a target possession retention of 0.90, has no solution in this model at any threshold, and the dribble-speed calibration failed against an absolute speed anchor before the ratio anchor worked. Both are documented in the calibration scripts.

3.3 Observation and action spaces

Each attacker observes a 120-dimensional vector: its own position and velocity, relative position, relative velocity and carrier flag for each of nine teammates, relative position and velocity for each of eleven defenders in fixed roster order, a seven-element ball block, its distance to the zone and the current zone control, an offside block, the normalized clock, four reserved constraint-rate slots, and a one-hot agent index. The one-hot is what lets ten agents under identical weights learn distinct roles.

Each attacker emits three discrete choices per tick: one of nine directions, eight compass headings plus stationary; one of three speed settings at 0.3, 0.6 and 1.0 of maximum; and one of ten ball

actions, hold or a pass to one of nine teammates. The joint action is therefore 30 discrete choices per tick, and they are correlated in two ways that matter. The ball dimension is only read for whichever agent currently holds the ball, and the other nine are masked to hold, so nine tenths of that dimension is inert on any given tick and which tenth is live changes with possession. And because the outcome depends on where all ten players are, the value of any one agent’s movement choice depends on the other nine, which the shared policy has to represent without communicating.

3.4 Reward and behavioral indicators

We use two behavioral indicators. The first, C_{slow} , fires for an individual attacker on a tick when its speed is below 2 m/s. Since the three speed settings correspond to 1.5, 3.0 and 5.0 m/s, this indicator distinguishes the lowest gear from the other two, modulo acceleration between settings. The second, C_{crowd} , fires when at least four attackers are within the zone radius of the zone centre.

The two differ in a way we return to in Sections 4.6 and 6.4. The slow indicator is computed per attacker, so each of the ten agents contributes its own cost signal on every tick and the batch average in Equation (4) runs over attacker-ticks. The crowding indicator is a team-level event. It is evaluated once per tick over all ten positions and the same scalar is then broadcast to every agent, so the average runs over ticks rather than attacker-ticks and every agent receives an identical cost regardless of whether it was personally inside the disc.

3.5 Simulator throughput

Pitch control is a large amount of small arithmetic evaluated at every grid cell for every player on every tick, across all parallel environments. In our setup it was the training bottleneck rather than the policy update. We reimplemented it as a CUDA kernel in raw C++ through CuPy’s `RawKernel` and kept the NumPy implementation as a reference rather than replacing it. The kernel gives 5.4 times the environment steps per second, and the two backends agree to within 9×10^{-4} on every output, checked against a stored fixture on every run rather than verified once. The experiments in this paper total roughly forty training runs at budgets between 2.5M and 10M steps, which the NumPy backend would not have made practical.

4 Method

We use the framework of Roy et al. [8] without modification to its constraint machinery. This section restates the parts we rely on, citing their equation numbers, and then lists the two places our implementation departs from theirs. Equations (1) through (7) are theirs.

4.1 Constrained objective

A constrained MDP [2] seeks a policy of maximal expected return among those satisfying a set of cost bounds. Following [8, Eq. 6],

$$\pi^* = \arg \max_{\pi \in \Pi} J_R(\pi) \quad \text{s.t.} \quad J_{C_k}(\pi) \leq d_k, \quad k = 1, \dots, K. \quad (1)$$

4.2 Indicator cost functions

Roy et al. restrict attention to cost functions that are indicators [8, Eq. 9],

$$C_k(s, a) = \mathbb{I}(\text{behavior } k \text{ is met in } (s, a)), \quad (2)$$

and observe that rewriting the discounted sum of such a cost as an expectation over the visitation distribution makes it a rescaled probability [8, Eqs. 10–13],

$$J_{C_k}(\pi) = Z(\gamma, T) \Pr_{(s,a) \sim x_\pi} [\text{behavior } k \text{ met in } (s, a)], \quad (3)$$

with $Z(\gamma, T) = \sum_{t=0}^T \gamma^t$. Dividing the constraint through by Z leaves a normalized threshold $\tilde{d}_k \in [0, 1]$ that reads as a desired behavior rate. This is what lets a designer write 0.75 and mean three quarters of ticks. In practice the constraint is estimated as a batch average [8, Eq. 14],

$$\tilde{J}_{C_k}(\pi) := \frac{1}{N} \sum_{i=1}^N C_k(s_i, a_i). \quad (4)$$

4.3 Multiplier normalisation

Unnormalized multipliers on a hard-to-satisfy constraint grow without bound and destabilize learning. Roy et al. normalize them with a softmax over base parameters z_k and a dummy variable a_0 [8, Eq. 15],

$$\lambda_k = \frac{\exp(z_k)}{\exp(a_0) + \sum_{k'=1}^K \exp(z_{k'})}, \quad \lambda_0 := 1 - \sum_{k=1}^K \lambda_k, \quad (5)$$

which gives the min-max problem [8, Eq. 16],

$$\max_{\pi} \min_{z_{1:K} \geq 0} \lambda_0 J_R(\pi) - \sum_{k=1}^K \lambda_k (J_{C_k}(\pi) - \tilde{d}_k). \quad (6)$$

4.4 Bootstrap constraint

Because $\lambda_0 = 1 - \sum_k \lambda_k$, the weight on the main objective is squeezed while the optimizer searches for a feasible policy, leaving little traction on the task. Roy et al. add a lower-bound success constraint indexed $K+1$ and lend its multiplier to the main objective [8, Sec. 4.3],

$$\tilde{\lambda}_0 := \max(\lambda_0, \lambda_{K+1}), \quad (7)$$

substituted for λ_0 in Equation (6). The handover from λ_{K+1} back to λ_0 appears as a step change in Figure 5, left panel.

4.5 Where our implementation differs

Their algorithm applies Equation (6) at the Q -values of $K+2$ separate critics. We apply it at generalized advantages, standardized per channel:

$$\hat{A} = \tilde{\lambda}_0 \bar{A}_R + \lambda_{K+1} \bar{A}_{K+1} - \sum_{k=1}^K \lambda_k \bar{A}_k, \quad (8)$$

where $\bar{A}$ denotes a batch-standardized advantage. This is the only structural change. The sign convention, the bootstrap term and the multiplier dynamics follow (6) and (7) directly. We compute cost advantages under a separate discount γ_k and scale their targets by $(1 - \gamma_k)$, which implements the $Z(\gamma, T)$ normalization of (3) for truncated returns.

Table 1: The two places our implementation departs from Roy et al. [8]. Equations (1)–(7) are used unmodified.

Deviation	Reason
On-policy PPO in place of off-policy SAC	Their Appendix A notes that the multiplier objective is optimized on-policy even inside an off-policy learner, and that reconciling the two forces a tradeoff between estimate variance and sample efficiency. PPO removes the tension at the cost of sample reuse.
Combination at standardized advantages rather than at Q -values	PPO has no Q function to combine at. Standardizing each advantage channel before combining keeps the terms on a common scale, which is the role their $K+2$ separate critics play.

4.6 Indicator granularity

A shared policy raises a design question that does not arise with a single agent: what unit does the indicator in Equation (2) count over? The same behavioral intent can be written as a per-agent rate or as a team-level rate, and the two give different learning problems while describing the same behavior. Roy et al. have no occasion to make this choice, since their agent acts alone.

We use per-agent indicators for C_{slow} . Section 6.4 shows what happens under a team-mean indicator at the same threshold.

4.7 Notation

Symbol	Meaning
J_R, J_{C_k}	Expected return; expected discounted cost k
$\tilde{d}_k$	Normalized target rate for constraint k
$\hat{J}_k$	Measured rate of behavior k
λ_k	Normalized multiplier on constraint k
$\lambda_0, \tilde{\lambda}_0$	Weight on task reward, before and after the bootstrap floor
$K+1$	Index of the success (bootstrap) constraint

5 Measurement

Frozen-policy probe against training logs. Averaging an indicator over training logs mixes behavior from every policy the run passed through. We instead evaluate a frozen final policy over 150,000 attacker-ticks. The two disagree: on the slow sweep, training logs report a mean absolute tracking error of 2.4 points against the probe’s 4.4. We report the probe figure throughout, since the log figure flatters the method.

Wilson intervals. The probe runs for a fixed number of ticks rather than a fixed number of episodes, so it cuts off mid-episode at whichever episode is running when the window ends. Short episodes are over-represented in what it captures, and short episodes are disproportionately turnovers. Counting every episode the probe touches therefore biases success downward. We report success only over episodes that terminated inside the window, with Wilson score intervals rather than normal approximations, since success rates near the ends of the range and modest episode counts are exactly where the normal interval misbehaves.

Table 2: Slow-constraint sweep, three seeds per target. Achieved rates track requested rates with slope 0.89. The seed spread in success is wide at the upper targets.

Requested $\tilde{d}$	Achieved $\hat{J}_{\text{slow}}$	Speed	Success
0.45	0.450 ± 0.039	2.117	0.730 ± 0.068
0.55	0.546 ± 0.002	1.920	0.839 ± 0.036
0.65	0.659 ± 0.032	1.674	0.686 ± 0.098
0.75	0.745 ± 0.011	1.507	0.688 ± 0.106
0.85	0.794 ± 0.011	1.377	0.640 ± 0.031

Tail means and what they hide. A run’s achieved rate is not a single number. The multiplier and the policy chase each other, so the measured rate oscillates, and reading the final logged point turns that oscillation into noise. We average the last 20% of logged points instead.

This convention hides something, and it is worth showing where. In sweep_crowd10_s3 at 2.5M steps the tail mean is 0.044, comfortably under its requested 0.10, so the run reads as feasible. Individual points inside that same window reach 0.112, above the threshold. Whether that run satisfied its constraint depends entirely on which summary is used, and neither summary is wrong. We use the tail mean throughout and note here that a feasibility call in this setting is a statement about a window, not about a policy.

6 Experiments

We report the experiments in the order we ran them, because the later ones exist to explain the earlier ones.

All runs use three seeds unless stated. Constrained runs train for 5M environment steps except the mechanism experiments in Section 6.4, which use 2.5M at two seeds. The unconstrained baseline is a single run at 10M steps on the identical task configuration, and it did not receive a hyperparameter search of its own; it inherits the settings the constrained runs use, minus the constraint machinery. Section 6.3 states the sample-efficiency comparison with that limitation attached.

6.1 One constraint transfers

The constraint tracks its target across the sweep (Table 2, Figure 2). A linear fit to the target means gives a slope of 0.89 with a mean absolute error of 1.5 points; the per-run frozen-probe error is 4.4 points. Tracking degrades at $\tilde{d} = 0.85$, where the requested rate approaches what the policy does unconstrained and there is little left for the constraint to do.

The constraint does not cost sample efficiency, and at the shorter budget it appears to buy some. Three constrained runs at $\tilde{d} = 0.75$ trained to 2.5M steps reach a mean tail success of 0.738, against 0.657 for an unconstrained run trained to 10M on the same task. Forcing the policy to hold a movement rate is a better training signal here than leaving it free, not a tax on performance.

We state this as approximate rather than as a clean fourfold saving, for two reasons that Section 6.3 makes concrete. The unconstrained baseline is one seed, and the three constrained runs at 2.5M span 0.622 to 0.805, so the gap is of the same order as the spread within either arm. And the comparison is again across budgets,

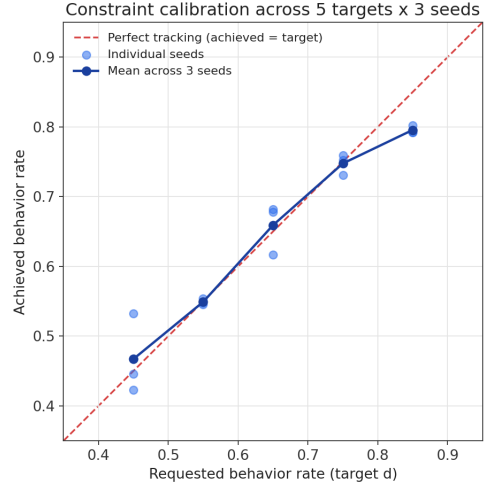


Figure 2: Achieved against requested rate for the slow constraint, five targets at three seeds. Tracking holds across most of the range and saturates at the top, where $\tilde{d} = 0.85$ approaches the unconstrained behavior rate.

which we show elsewhere in this paper to be unreliable in this environment: the same constrained configuration trained to 5M reaches 0.623, below the unconstrained 10M baseline rather than above it. The honest version of this result is that a constrained policy at 2.5M is at least competitive with an unconstrained one at 10M, and that the ordering does not survive extending the constrained run.

Two features of Table 2 are worth stating plainly. Success declines as the requested rate rises, which is expected. And the seed spread in success reaches 0.106 at $\tilde{d} = 0.75$, wider than the gap between adjacent targets. Any claim about how a target rate affects task performance in this environment needs more than three seeds to support it.

6.2 A constraint that never binds

We had observed crowding rise under the slow constraint and added C_{crowd} as a second constraint to suppress it. Across nine runs covering three thresholds, the achieved crowding rate collapsed toward zero regardless of which target we requested. Requesting 0.05, 0.10 and 0.15 all produced final rates between 0.011 and 0.044. The dial behaved as a switch.

To separate an interaction between the two constraints from a property of the crowding constraint itself, we ran the crowding constraint alone with the slow constraint disabled. Figure 3 shows the result and explains it. The measured crowding rate at initialization is between 0.000 and 0.012, below every threshold we swept. The constraint is satisfied before training begins. In the six runs at $\tilde{d} = 0.10$ and $\tilde{d} = 0.15$ the multiplier peaks at its initialization value and decays monotonically, never responding to a violation because none occurs. Only at $\tilde{d} = 0.05$ does the rate cross the threshold, transiently, between 1M and 2M steps, and there the multiplier does activate.

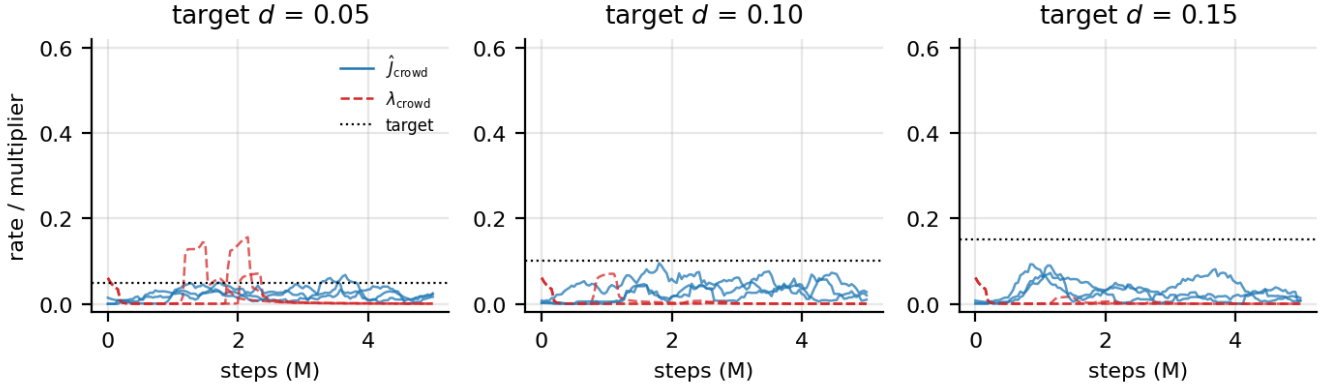


Figure 3: The crowding constraint alone, three thresholds at three seeds each, with the slow constraint disabled. The measured rate (blue) starts below the requested target (dotted) and stays there. The multiplier (red, dashed) decays from its initialization without responding to a violation, except in the $\tilde{d} = 0.05$ panel where the rate crosses the threshold between 1M and 2M steps.

An assumption stated in passing. Roy et al. are explicit that the semantics of their thresholds hold assuming that the constraint is binding and that a feasible policy is found [8, Sec. 4.1]. The first half of that assumption is not tested in their work, and to our knowledge has not been tested since. Our nine runs are what its failure looks like.

A reader may note that an inactive constraint carries a zero multiplier by construction, and that this is elementary. We agree, and the finding is not about the optimizer. It is about the workflow the framework exists to support. A practitioner observes an unwanted behavior, picks a target rate below what they measured, and adds a constraint. That workflow fails silently here, because the behavior they measured is not the behavior the constraint sees during training. A Lagrangian constraint can only pull a rate toward its target from the violating side. When the policy sits on the satisfied side, the specification has no effect and produces no error, no warning and no visible difference from a run where the constraint was never added.

Which measurement saw the side effect. The crowding that prompted the second constraint was measured with the frozen-policy probe on a slow-constrained run trained to 2.5M steps, where it reached 0.211 against an unconstrained level near 0.029. The training logs of the slow-constrained runs at 5M do not show elevation on that scale, ending between 0.014 and 0.079.

We cannot resolve this with the runs we have. The two numbers come from different instruments, and Section 5 already shows those instruments disagreeing by two points on a quantity where they overlap, so a disagreement here is not surprising on its own. It is also possible that the crowding is real at 2.5M and resolves under a longer budget, which the logs are consistent with and which the probe was never run to check. Distinguishing the two requires the probe on the 5M checkpoints and nothing else. We record it as an open question rather than picking the reading that suits the argument.

Table 3: Tail-mean success by arm. The 2.5M two-constraint baseline was our original comparator and sits 11 points above the same configuration trained to 5M.

Arm	n	Success
Two-constraint, 2.5M	3	0.738
Two-constraint, 5M	3	0.623
Crowd only, 5M	9	0.635
Slow + crowd, 5M	9	0.686

6.3 A cost that was not there

An earlier version of this work reported that adding the crowding constraint cost between 13 and 17 points of task success. That comparison was between crowd-constrained runs trained to 5M steps and a two-constraint baseline trained to 2.5M. We ran the baseline again at a matched budget with three seeds.

At 5M the two-constraint baseline reaches 0.623 and the stacked runs reach 0.686 (Table 3, Figure 4). The stacked arm sits above the baseline. The reported cost was an artifact of the budget mismatch.

Two mechanisms account for it, and both are worth naming because the original design was a natural one. First, seed variance at 2.5M is large: two seeds of the identical configuration tail at 0.805 and 0.622, an 18-point spread, and our original single-seed baseline sat near the top of that range. Second, the learning rate anneals linearly to zero over the configured budget, so at step 2.4M the 2.5M run is at 1.9×10^{-5} while the 5M run is at 1.6×10^{-4} . The shorter run has effectively converged at the point where the comparison was drawn. The two budgets are not comparable at any shared step, and the right panel of Figure 4 shows that the 2.5M run is not an early segment of the 5M trajectory.

6.4 Mechanism experiments

We run three variants at 2.5M steps with two seeds each, all at $\tilde{d}_{\text{slow}} = 0.75$, against a control using the full framework. Two of the three are replications of results in the original work, carried into our regime; the third has no single-agent counterpart. Two

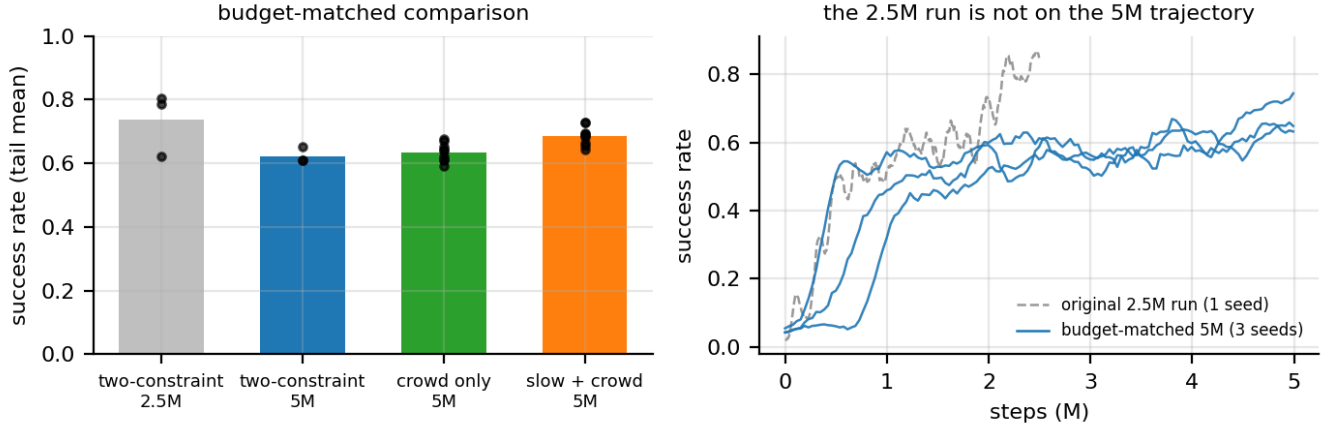


Figure 4: Left: tail-mean success by arm, individual runs overlaid. At a matched 5M budget the stacked arm sits above the two-constraint baseline rather than below it. Right: the original 2.5M run against the three budget-matched 5M runs of the same configuration. The shorter run is not an early point on the longer trajectory.

Table 4: Mechanism experiments, two seeds each, 2.5M steps, $\tilde{a}_{\text{slow}} = 0.75$. Per-seed values are shown because the seeds disagree in two of the four arms.

Arm	$\hat{J}_{\text{slow}}$	λ_0	Success
Control	0.750 / 0.778	0.976 / 0.974	0.805 / 0.622
No bootstrap	0.722 / 0.702	0.983 / 0.990	0.629 / 0.621
Unnorm. λ	0.770 / 0.746	1.000 / 1.000	0.632 / 0.532
Team-mean ind.	0.626 / 0.840	0.007 / 0.003	0.562 / 0.495

seeds cannot establish a rate of failure, and we report these as observations rather than measured frequencies.

Indicator granularity is load-bearing (new). Replacing the per-agent slow indicator with a team-mean indicator at the same threshold fails in both seeds and fails in a specific way (Figure 5). The task-reward weight λ_0 collapses to 0.007 and 0.003, leaving under one percent of the objective on the task, and unlike the control it never recovers through the bootstrap handover. The achieved rate oscillates between 0.5 and 1.0 rather than settling, ending at 0.626 and 0.840 against a requested 0.75. Success falls to 0.562 and 0.495, and mean episode length roughly halves, from 108 and 95 ticks to 54 and 57. A constraint describing the same behavior at the same threshold becomes unsatisfiable when counted over the wrong unit.

Unnormalized multipliers (replication). Roy et al. motivate Equation (5) with a scenario in which an unsatisfiable constraint drives its multiplier without bound until the critic loss diverges and performance collapses [8, Fig. 3]. Replacing the softmax with independent non-negative multipliers reproduces this in one seed of two: λ_{K+1} reaches 30.2 and λ_{slow} reaches 1.40, success falls to 0.532, and episode length collapses to 60 ticks. The other seed is indistinguishable from control. Their demonstration used a deliberately infeasible constraint; ours arises without one, which suggests the

instability can appear in ordinary training, though we cannot say how often.

Removing the bootstrap has no effect here (replication, negative). Roy et al. show that without the bootstrap constraint, an agent under five simultaneous constraints settles on a trivial constraint-satisfying behavior and never solves the main task [8, Fig. 5b]. We do not reproduce this. Achieved rates are 0.722 and 0.702 against control’s 0.750 and 0.778, success is 0.629 and 0.621 against control’s 0.805 and 0.622, and episode lengths are comparable.

The likely reason is consistent with their account rather than against it. The bootstrap exists because $\lambda_0 = 1 - \sum_k \lambda_k$ is squeezed toward zero as constraints accumulate. Our variant runs one behavioral constraint plus success, and λ_0 sits at 0.98 throughout, so there is no squeeze to relieve. We read this as evidence that the mechanism’s necessity scales with constraint count and that it is inert at $K = 1$, not as a failure of the mechanism.

7 Discussion

Our two failures point at the same thing from opposite directions. A target rate specifies a behavior only when two conditions hold.

The first is the binding assumption Roy et al. state in passing [8, Sec. 4.1]: the policy has to be on the violating side of the threshold, at least some of the time, for the multiplier to have anything to respond to. Our crowding constraint failed this from initialization onward. The second is that the indicator has to count over a unit that gives the optimizer a usable gradient. Our team-mean variant failed this while describing exactly the behavior we intended.

Neither condition is visible from the target rate. A practitioner reading $\tilde{a}_{\text{crowd}} = 0.10$ and $\tilde{a}_{\text{slow}} = 0.75$ has no way to tell that the first is inert and the second is not, and neither run reports an error. Both are cheap to check before spending compute. Measure the indicator on the untrained policy and confirm the requested target sits below it. Check that the indicator’s baseline rate is not saturated near zero or one, where it has little room to move. Both cost a single evaluation pass.

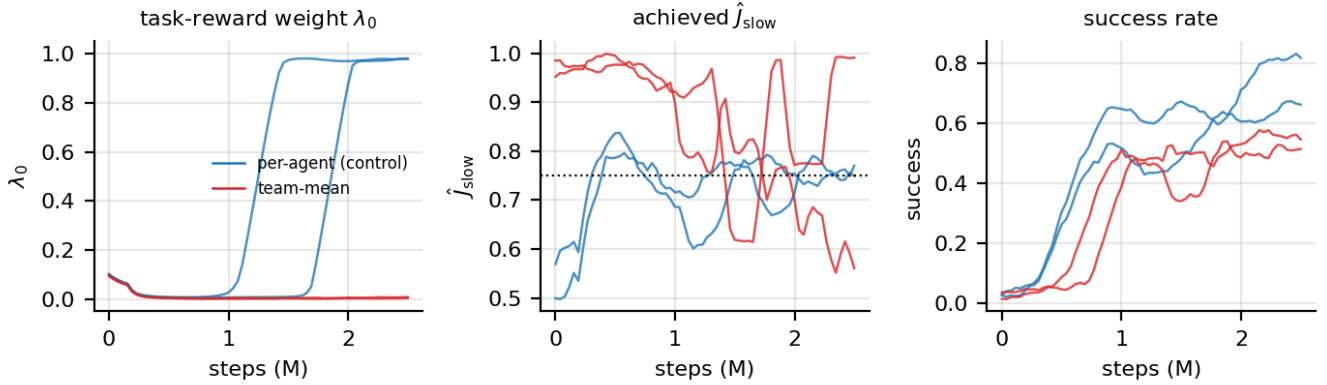


Figure 5: Aggregating the slow indicator over the team rather than per agent. Left: the task-reward weight collapses and never recovers, while the control hands over from λ_{K+1} to λ_0 at 1.2M and 1.8M steps. Center: the achieved rate oscillates across the full range instead of settling at the requested 0.75. Right: success is 10 to 13 points below control.

Direct behavior specification does transfer to a harder regime than the one it was built for. What does not transfer automatically is the assumption that writing down a rate is the same as specifying a behavior.

8 Limitations

Two seeds on the mechanism experiments. Section 6.4 rests on two seeds per arm. The team-mean result is consistent across both and comes with a mechanism visible in the traces, which we think makes it credible, but the unnormalized-multiplier result splits and we cannot report a failure rate for it.

Two measurements disagree about crowding. The elevated crowding that motivated the second constraint was measured with a frozen-policy probe on a 2.5M run. Training-log values on the 5M runs do not show the same elevation. We do not know whether crowding resolves with a longer budget or whether the two instruments disagree, and we cannot distinguish these without running the probe on the 5M checkpoints. That evaluation is the resolving experiment and we did not have the compute for it.

Seed variance in the slow sweep. At $\tilde{d} = 0.75$ the seed spread in success is 0.106, wider than the interval between adjacent targets. Conclusions about how the requested rate trades against task performance are correspondingly weak.

At most two constraints. Roy et al. handle up to five simultaneous constraints, and their bootstrap mechanism exists for that setting. Our experiments use at most two, so we say nothing about how either of our conditions behaves as the constraint set grows.

Single task and single zone. Every result comes from one scenario with one target zone. We do not know how much of the tracking behavior depends on that geometry.

9 Conclusion

Direct behavior specification via constrained reinforcement learning transfers to a shared-policy multi-agent setting trained on-policy against an adaptive opponent, for a constraint the policy

actually violates. It does nothing for a constraint the policy already satisfies, and it breaks when the indicator counts over the team rather than the agent. Both conditions are checkable in advance and neither is apparent from the specification. We also report a 15-point effect that disappeared under a budget-matched baseline, because the comparison that produced it is one others are likely to draw.

Acknowledgments

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained Policy Optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 22–31.
- [2] Eitan Altman. 1999. *Constrained Markov Decision Processes*. Vol. 7. CRC Press.
- [3] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).
- [4] Shangding Gu, Jakub Grudzien Kuba, Yuanpei Chen, Yali Du, Long Yang, Alois Knoll, and Yaodong Yang. 2023. Safe Multi-Agent Reinforcement Learning for Multi-Robot Control. *Artificial Intelligence* 319 (2023), 103905.
- [5] Victoria Krakovna, Laurent Orseau, Ramana Kumar, Miljan Martić, and Shane Legg. 2018. Penalizing Side Effects using Stepwise Relative Reachability. *arXiv preprint arXiv:1806.01186* (2018).
- [6] Karol Kurach, Anton Raichuk, Piotr Stańczyk, Michał Zajac, Olivier Bachem, Lasse Espeholt, Carlos Riquelme, Damien Vincent, Marcin Michalski, Olivier Bousquet, and Sylvain Gelly. 2020. Google Research Football: A Novel Reinforcement Learning Environment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4501–4510.
- [7] Chenyi Liu, Nan Geng, Vaneet Aggarwal, Tian Lan, Yuan Yang, and Mingwei Xu. 2021. CMIX: Deep Multi-Agent Reinforcement Learning with Peak and Average Constraints. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*.
- [8] Julien Roy, Roger Girgis, Joshua Romoff, Pierre-Luc Bacon, and Christopher Pal. 2022. Direct Behavior Specification via Constrained Reinforcement Learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)* (PMLR, Vol. 162).
- [9] William Spearman. 2018. Beyond Expected Goals. In *MIT Sloan Sports Analytics Conference*.
- [10] William Spearman, Austin Basye, Greg Dick, Ryan Hotovy, and Paul Pop. 2017. Physics-Based Modeling of Pass Probabilities in Soccer. In *MIT Sloan Sports Analytics Conference*.
- [11] Adam Stooke, Joshua Achiam, and Pieter Abbeel. 2020. Responsive Safety in Reinforcement Learning by PID Lagrangian Methods. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. 9133–9143.
- [12] Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. 2019. Reward Constrained Policy Optimization. *International Conference on Learning Representations (ICLR)* (2019).

- [13] Alexander Matt Turner, Dylan Hadfield-Menell, and Prasad Tadepalli. 2020. Conservative Agency via Attainable Utility Preservation. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*.
- [14] Zhe Wang, Petar Veličković, Daniel Hennes, Nenad Tomašev, Laurel Prince, Michael Kaisers, Yoram Bachrach, Romuald Elie, Li Kevin Wenliang, Federico Piccinini, William Spearman, Ian Graham, Jerome Connor, Yi Yang, Adrià Recasens, Mina Khan, Nathalie Beauguerlange, Pablo Sprechmann, Pol Moreno, Nicolas Heess, Michael Bowling, Demis Hassabis, and Karl Tuyls. 2024. TacticAI: An AI Assistant for Football Tactics. *Nature Communications* 15, 1 (2024), 1906.